AI505

Optimization

Discrete Optimization
Integer Linear Programming and Branch and Bound

Marco Chiarandini

Department of Mathematics & Computer Science
University of Southern Denmark

Discrete Optimization

- Discrete optimization is a branch of optimization that deals with problems where the solution space is discrete, meaning that the **variables** can only take on **specific, distinct values**.
- This is in contrast to continuous optimization, where the variables can take on any value within a given range.
- Discrete optimization problems are often NP-hard, meaning that they are computationally challenging to solve.
- It is important to develop efficient algorithms and heuristics to solve these problems, as they often arise in real-world applications.
- Discrete optimization is widely used in various fields, including operations research, computer science, engineering, and economics.

Discrete Optimization

- What is discrete optimization (previous slide)? And Combinatorial Optimization?
- Integer (linear) programming
- Exact solution techniques
  - Dedicated algorithms
  - Dynamic programming
  - Constraint programming
  - SAT solvers
- Approximate solution techniques
  - Approximation algorithms
  - Greedy algorithms and Local Search
  - Metaheuristics, Randomized Optimization Algorithms
  - Matheuristics
  - ...

Discrete vs Combinatorial Optimization

In Combinatorial Optimization, variables have some combinatorial structure, ie, sets, permutations, paths, etc.

Definition (Combinatorial Optimization Problem (COP))

Input: Given a finite set $N = \{1, \ldots, n\}$ of objects,
weights c_j for all $j \in N$,
a collection $\mathcal{F}$ of feasible subsets of N
Task: Find a minimum weight feasible subset, ie,

$$\underset{S \subseteq N}{\text{minimize}} \left\{ \sum_{j \in S} c_j \mid S \in \mathcal{F} \right\}$$

COP can also be modelled as discrete optimization problems.

Typically: **incidence vector** of S, $\boldsymbol{x}^S \in \mathbb{B}^n$: $x_j^S = \begin{cases} 1 & \text{if } j \in S \\ 0 & \text{otherwise} \end{cases}$

Discrete (or Combinatorial) Optimization

Solving problems with variables that are discrete instead of continuous

Example

- Set covering and set partitioning
- Knapsack problem
- Traveling salesman problem
- Vehicle routing problem
- Job scheduling problem
- Facility location problem
- Bin packing problem
- Graph coloring problem
- Maximum flow problem
- Minimum spanning tree problem
- Shortest path problem
- Steiner tree problem
- Hamiltonian path problem

The set of possible discrete values can be finite or infinite

Solution Methods

Exact Methods:

- Integer programming
- Combinatorial optimization algorithms
- Graph theory algorithms
- Scheduling algorithms
- SAT
- Dedicated Branch and bound
- Dynamic programming
- Constraint programming
- Integer linear programming
- Mixed-integer programming
- Network flow algorithms

Heuristic Methods:

- Approximation algorithms
- Greedy algorithms
- Local Search algorithms
- Metaheuristics
  - Genetic algorithms
  - Simulated annealing
  - Ant colony optimization
  - Tabu search
  - ...

Outline

1. Integer Linear Programming

Notation: Set of Integer Numbers

- $\mathbb{Z}$ set of integer numbers $\{\ldots, -3, -2, -1, 0, 1, 2, 3, \ldots\}$
- $\mathbb{Z}^+$ set of positive integers
- $\mathbb{Z}_0^+$ set of nonnegative integers ($\{0\} \cup \mathbb{Z}^+$)
- $\mathbb{N}_0$ set of natural numbers, ie, nonnegative integers $\{0, 1, 2, 3, 4, \ldots\}$
- $\mathbb{B}$ set of binary numbers

Mixed Integer Linear Programming (ILP)

Linear Objective • Linear Constraints • but! integer variables

$$\begin{aligned} \max \quad & \boldsymbol{c}^T \boldsymbol{x} \\ A\boldsymbol{x} &\leq \boldsymbol{b} \\ \boldsymbol{x} &\geq \boldsymbol{0} \end{aligned}$$

Linear Programming (LP)

$$\begin{aligned} \max \quad & \boldsymbol{c}^T \boldsymbol{x} \\ A\boldsymbol{x} &\leq \boldsymbol{b} \\ \boldsymbol{x} &\geq \boldsymbol{0} \\ \boldsymbol{x} & \text{ integer} \end{aligned}$$

Integer Linear Programming (ILP)

$$\begin{aligned} \max \quad & \boldsymbol{c}^T \boldsymbol{x} \\ A\boldsymbol{x} &\leq \boldsymbol{b} \\ \boldsymbol{x} &\in \{0,1\}^n \end{aligned}$$

Binary Integer Program (BIP)
0/1 Integer Programming

$$\begin{aligned} \max \quad & \boldsymbol{c}^T \boldsymbol{x} + \boldsymbol{h}^T \boldsymbol{y} \\ A\boldsymbol{x} + G\boldsymbol{y} &\leq \boldsymbol{b} \\ \boldsymbol{x} &\geq \boldsymbol{0} \\ \boldsymbol{y} &\geq \boldsymbol{0} \\ \boldsymbol{y} & \text{ integer} \end{aligned}$$

Mixed Integer Linear Programming (MILP)

$$\begin{aligned} \max \quad & f(\boldsymbol{x}) \\ g(\boldsymbol{x}) &\leq \boldsymbol{b} \\ \boldsymbol{x} &\geq \boldsymbol{0} \\ \boldsymbol{x} & \text{ integer} \end{aligned}$$

Integer Non-linear Programming (MINLP)

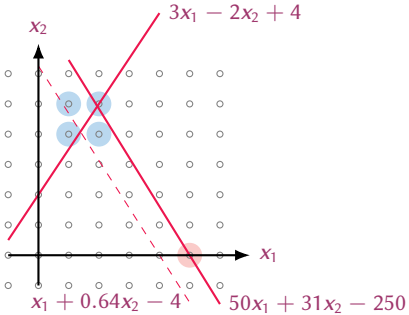
Mathematical Programming: Modeling

- Find out exactly what the decision maker needs to know:
  - which investment?
  - which product mix?
  - which job j should a person i do?
- Define **Decision Variables** of suitable type (continuous, integer valued, binary) corresponding to the needs and **Known Parameters** corresponding to given data.
- Formulate **Objective Function** computing the benefit/cost
- Formulate mathematical **Constraints** indicating the interplay between the different variables.

Rounding

$$\begin{aligned} \max\ 100x_1 &+ 64x_2 \\ 50x_1 &+ 31x_2 \leq 250 \\ 3x_1 &- 2x_2 \geq -4 \\ x_1, x_2 &\in \mathbb{Z}_0^+ \end{aligned}$$

LP optimum $(376/193, 950/193)$
IP optimum $(5, 0)$



Note: rounding does not help in the example above!

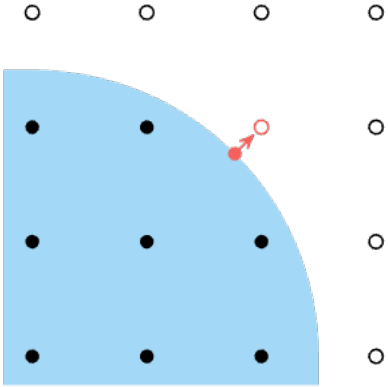
$\rightsquigarrow$ feasible region convex but not continuous: Now the optimum can be on the border (vertices) but also **internal**.

Possible way: solve the **relaxed** problem.

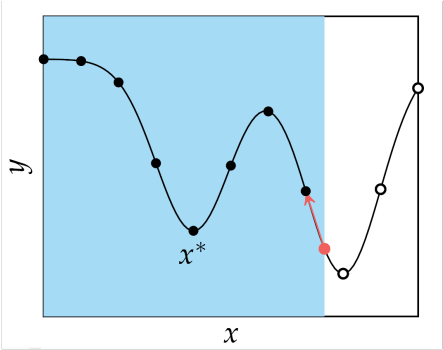
- If solution is **integer**, done.
- If solution is **rational** (never irrational) try rounding to the nearest integers (but may exit feasibility region)
  if in $\mathbb{R}^2$ then 2^2 possible roundings (up or down)
  if in $\mathbb{R}^n$ then 2^n possible roundings (up or down)

Rounding

Rounding to infeasible point



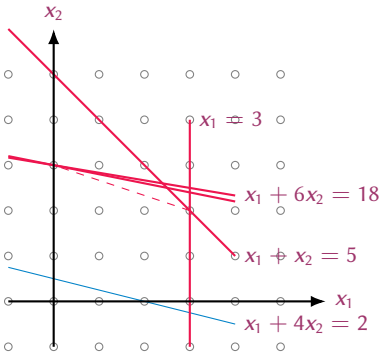
Rounding to suboptmal point



If A is integral, the error of a rounded solution can be bounded

Cutting Planes

$$
\begin{array}{lrcl}
\max & x_1 + 4x_2 & & \\
 & x_1 + 6x_2 & \leq & 18 \\
 & x_1 & \leq & 3 \\
 & x_1, x_2 & \geq & 0 \\
 & x_1, x_2 \text{ integers} & &
\end{array}
$$



Cuts: Useful valid inequalities: they are satisfied by all integer solutions but not by all relaxed solutions

Cutting Plane Method

- The cutting plane method solves the relaxed LP, adds linear constraints, then repeats until the solution is exact
- Solves mixed integer programs exactly
- The linear constraints are chosen such that all discrete points are still feasible, but the relaxed solution is not

Chvatal-Gomory's Cutting Plane Algorithm

- Recall that we can partition a vertex (and also an optimal one $\boldsymbol{x}^*$) as

$$A_B\boldsymbol{x}_B^* + A_N\boldsymbol{x}_N^* = \boldsymbol{b} \quad \Longrightarrow \quad \boldsymbol{x}_B^* = A_B^{-1}\boldsymbol{b} - A_B^{-1}A_N\boldsymbol{x}_N^*$$

$$\boldsymbol{x}_B^* + \lfloor A_B^{-1}A_N\rfloor\boldsymbol{x}_N^* \leq A_B^{-1}\boldsymbol{b}$$

$$\boldsymbol{x}_B^* + \lfloor A_B^{-1}A_N\rfloor\boldsymbol{x}_N^* \leq \lfloor A_B^{-1}\boldsymbol{b}\rfloor$$

$$A_B^{-1}\boldsymbol{b} - \lfloor A_B^{-1}\boldsymbol{b}\rfloor - (A_B^{-1}A_N - \lfloor A_B^{-1}A_N\rfloor)\boldsymbol{x}_N^* \leq 0$$

- Using this derivation, we can add an inequality constraint for every variable $u \in B$ that has a fractional value in the relaxed solution $\boldsymbol{x}^*$ (u is the index of the variable and of the row in A_B)

$$x_u^* - \lfloor x_u^*\rfloor - \sum_{j\in N}\left(\bar{A}_{uj} - \lfloor\bar{A}_{uj}\rfloor\right)x_j \leq 0 \qquad \bar{A} = A_B^{-1}A_N$$

- This "cuts out" the relaxed solution $\boldsymbol{x}^*$

$$\underbrace{x_u^* - \lfloor x_u^*\rfloor}_{>0} - \underbrace{\sum_{j\in N}\left(\bar{A}_{uj} - \lfloor\bar{A}_{uj}\rfloor\right)x_j^*}_{=0} > 0$$

Branch and Bound

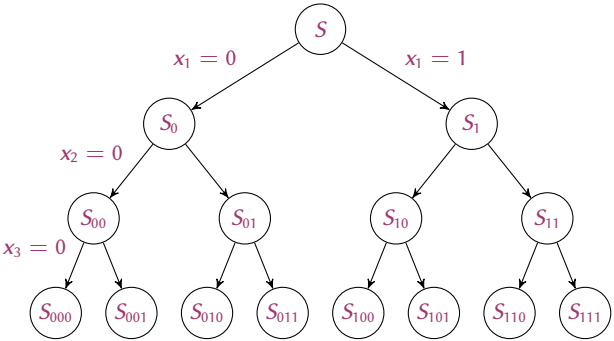
- General Branch and Bound: dedicated algorithms for specific problems (e.g., TSP, Knapsack, etc.)
- LP Based Branch and Bound for ILP: general method for solving integer linear programs
- (Spacial Branch and Bound for MINLP: general method for solving mixed integer nonlinear programs)

General Branch and Bound

- Consider the problem $z = \min\{\boldsymbol{c}^T\boldsymbol{x} : \boldsymbol{x} \in S\}$

- Divide and conquer: let $S = S_1 \cup \ldots \cup S_k$ be a decomposition of S into smaller sets, and let $z^k = \min\{\boldsymbol{c}^T\boldsymbol{x} : \boldsymbol{x} \in S_k\}$ for $k = 1, \ldots, K$. Then $z = \min_k z^k$

For instance if $S \subseteq \{0,1\}^3$ the enumeration tree is:

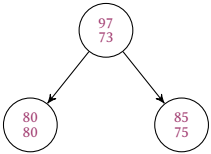


Bounding

Let's consider a minimization problem

- Let $\underline{z}^k$ be a **lower bound** on z^k (dual bound)
- Let $\overline{z}^k$ be an **upper bound** on z^k (primal bound)
- $(\underline{z}^k \leq z^k \leq \overline{z}^k)$
- $\underline{z} = \min_k \underline{z}^k$ is a **lower bound** on z
- $\overline{z} = \min_k \overline{z}^k$ is an **upper bound** on z

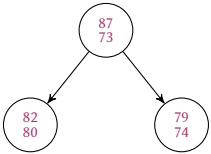
Pruning (Fathoming)



$\overline{z} = 80$

$\underline{z} = 75$

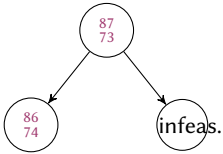
pruned by optimality



$\overline{z} = 79$

$\underline{z} = 74$

pruned by bounding

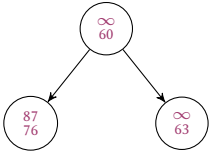


$\overline{z} = 86$

$\underline{z} = 74$

pruned by infeasibility

Pruning



$\overline{z} = 76$

$\underline{z} = 63$

nothing to prune

LP Based Branch and Bound

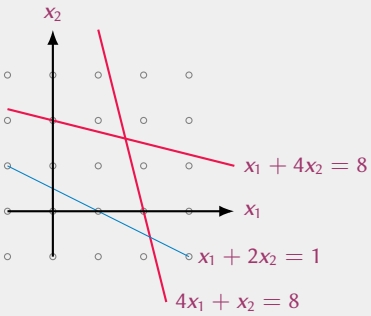
- Algorithm for efficiently searching the very large set of solution possibilities (first proposed by Ailsa Land and Alison Doig, “An automatic method of solving discrete programming problems”, 1960)
- **Branching** is dividing the domain into sections
- **Bounding** is keeping track of the best solution so far and rejecting regions that cannot improve upon it
- In the worst case, the algorithm has to search all possibilities but in practice it works very well. Combined with cutting planes, this approach forms the basis of many commercial MIP solvers.

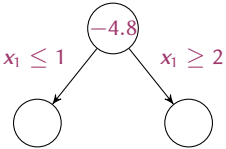
LP Based Branch and Bound

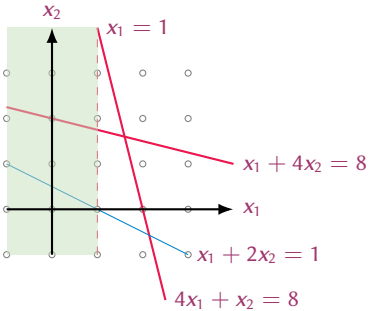
LP based branch and bound uses the optimal value of the LP relaxation problem as dual bound

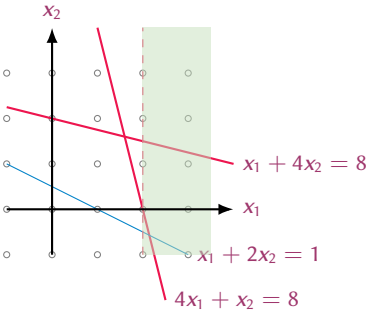
Example

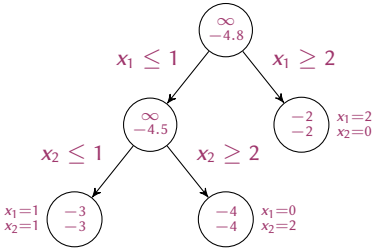
$$
\begin{array}{rrcrcl}
\min & -x_1 & - & 2x_2 & & \\
 & x_1 & + & 4x_2 & \leq & 8 \\
 & 4x_1 & + & x_2 & \leq & 8 \\
 & & & x_1, x_2 & \geq & 0, \text{integer}
\end{array}
$$

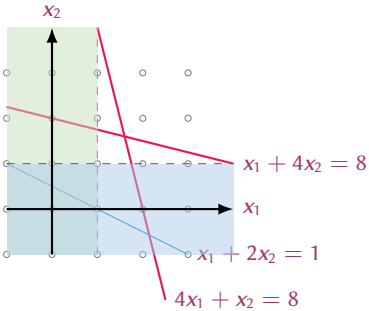


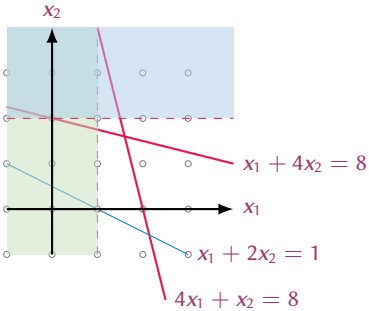

-4.8
$x_1 \leq 1$
$x_1 \geq 2$


x_2
$x_1 = 1$
$x_1 + 4x_2 = 8$
x_1
$x_1 + 2x_2 = 1$
$4x_1 + x_2 = 8$


x_2
$x_1 + 4x_2 = 8$
x_1
$x_1 + 2x_2 = 1$
$4x_1 + x_2 = 8$


∞
-4.8
$x_1 \leq 1$
$x_1 \geq 2$
∞
-4.5
-2
-2
$x_1=2$
$x_2=0$
$x_2 \leq 1$
$x_2 \geq 2$
$x_1=1$
$x_2=1$
-3
-3
-4
-4
$x_1=0$
$x_2=2$


x_2
$x_1 + 4x_2 = 8$
x_1
$x_1 + 2x_2 = 1$
$4x_1 + x_2 = 8$


x_2
$x_1 + 4x_2 = 8$
x_1
$x_1 + 2x_2 = 1$
$4x_1 + x_2 = 8$

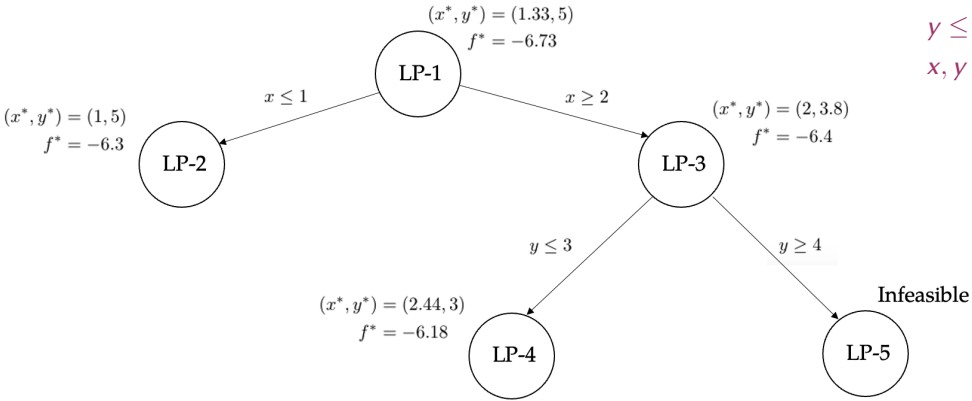
Branch and Bound: Pruning

$$\underset{x,y}{\text{minimize}} \; -1.3x - y$$

$$1.8x + y \leq 7.4$$

$$y \leq 5$$

$$x, y \in \mathbb{Z}$$



Pruning by: integrality, bounding, infeasibiliy.

Summary

- Discrete optimization problems require that the design variables be chosen from discrete sets.
- Many combinatorial optimization problems can be framed as an integer program, which is a linear program with integer constraints.
- In ILP, LP Relaxation, in which the continuous version of the discrete problem is solved, is by itself an unreliable technique for finding an optimal discrete solution but is central to more sophisticated algorithms.
- Both the cutting plane and branch and bound methods can be used to solve integer programs efficiently and exactly. The branch and bound method is quite general and can be applied to a wide variety of discrete optimization problems.